Big Data

Ecosystem and Tools

From Ingestion to Visualization

🎭 A Story Before We Begin

It's 2004. You work at a small company. Your database fits on one server. Your analyst runs a query every morning, gets the report, emails it to the CEO. Life is simple.

Fast forward to 2008. Facebook has 100 million users. Twitter is born. YouTube is serving millions of videos a day. Every click, every like, every share — all of it is data. The single server is gone. The morning query takes 3 days to run. The analyst is in tears.

Nobody planned for this. The tools weren't built for it. Engineers everywhere face the same wall — we have more data than we know what to do with, and nothing in our toolkit was designed for this scale.

That's the moment Big Data stops being a buzzword and becomes a real engineering problem. And that's exactly what this course is about — the ecosystem of tools that humans built, one problem at a time, to tame the data explosion.

Quick Recap — What is Big Data?

"Big data represents collections of data characterized by such large volume, velocity and variety that their transformation into usable value requires specific technologies and analytical methods"

VOLUME

No sampling — we observe and measure everything

VELOCITY

Data and results are often available in real time

VARIETY

Text, photos, audio, video — multiple sources merged

And then came more Vs...

- Veracity — Accuracy and trustworthiness of data
- Value — The business worth of the data collected
- Variability — The ways in which big data can be used and formatted

Why Do We Need New Tools?

- Exponential growth of unstructured data
  - Email, chat, blogs, social media, photos, videos, IoT sensors...
- Existing technologies were not designed for this
  - Relational databases, mainframes, spreadsheets — all built for structured, bounded data
- Parallel processing became accessible
  - Using multiple commodity machines in parallel instead of one expensive server
- New technologies emerged to solve these problems
  - Google File System (2003) → MapReduce (2004) → Hadoop (2006) → Spark (2014) → ...

Big Data Ecosystem — Course Roadmap

Each layer solves a specific problem. We will tackle them one by one.

INGESTION

Kafka · Sqoop · Flume · NiFi

→

STORAGE

HDFS · Delta Lake · Iceberg · S3

→

PROCESSING

Spark · Flink · MapReduce

→

VISUALIZATION

ELK · Tableau · Power BI · APIs

+ Modern Data Engineering layer: dbt · Apache Airflow · The Modern Data Stack

How the Layers Connect

Think of it as a data pipeline — raw data enters on the left, insights come out on the right

Raw Data

Clicks · Logs · Sensors · Social Media

▶

Ingest

Kafka
Flume
Sqoop

▶

Store

HDFS
Delta Lake
S3

▶

Process

Spark
Flink
dbt

▶

Visualize

Kibana
Tableau
APIs

Section 2

Ingestion Layer

The Problem of Getting Data In

🎭 The Story of Kafka — Born Out of Pain

It's 2011. LinkedIn is growing at over 1 million new users every week. Data is flying everywhere — user activity, messages, job views, connection requests. Their systems need to talk to each other constantly.

The problem? Their traditional message queues are crumbling under the load. Data gets lost. Systems lag behind. Engineers spend nights firefighting instead of building.

A team of engineers asks a different question: what if instead of thinking about messages, we think about a distributed commit log — an append-only, ordered, replayable stream of events?

They build it. They name it Kafka — after the author Franz Kafka — because the system was, in their own words, 'optimized for writing'. It gets open-sourced in 2011. Today, Kafka processes trillions of events per day at companies like Netflix, Uber, and Airbnb.

The Ingestion Problem

Before we can store or process data, we need to get it in. That is harder than it sounds.

BATCH INGESTION

- Data collected and moved in chunks at scheduled intervals
- Think: nightly ETL jobs, daily database exports
- Suitable for historical analysis and reporting
- Tools: Apache Sqoop, Apache Flume
- Limitation: not real-time — data is always 'old'

VS

STREAM INGESTION

- Data flows continuously in real time as events happen
- Think: user clicks, sensor readings, financial ticks
- Enables real-time decisions and live dashboards
- Tools: Apache Kafka, Apache NiFi
- Limitation: higher complexity to design and operate

Batch Ingestion — Apache Sqoop

Sqoop = SQL + Hadoop. A tool designed to transfer bulk data between relational databases (RDBMS) and HDFS.

RDBMS

MySQL · Oracle
PostgreSQL · SQL Server

▶

SQOOP

Import / Export
MapReduce jobs

▶

HDFS / Hive

Distributed Storage
Data Warehouse

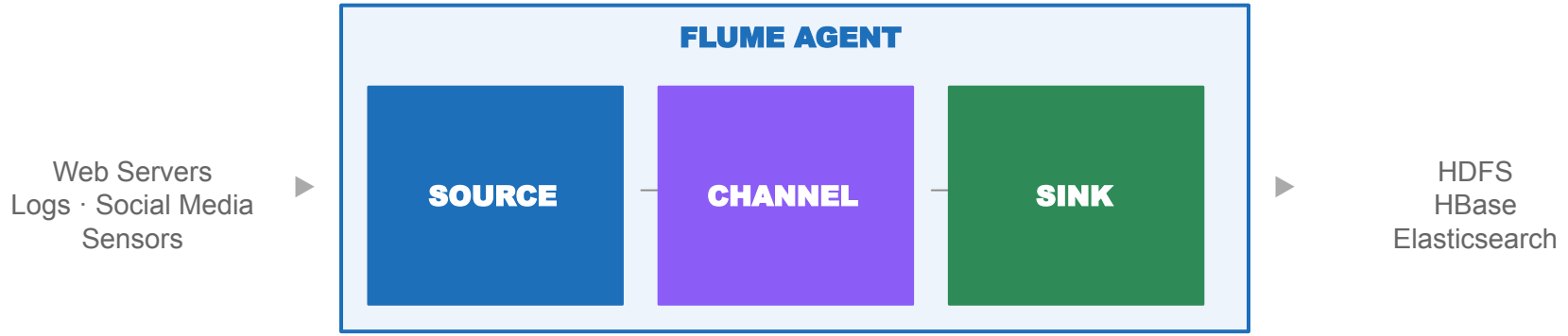
Key Features

- Bidirectional: import data from RDBMS into HDFS, or export from HDFS back to RDBMS
- Parallel transfer: uses multiple MapReduce mappers to speed up data movement
- Supports incremental loads: only transfer new or changed records since the last import
- Integrates directly with Hive and HBase for seamless data warehouse loading

Batch Ingestion — Apache Flume

Flume is a distributed service for collecting, aggregating and moving large amounts of log data and event data into HDFS.

Flume Agent Architecture



Key Features

- Designed for high-volume log and event data: web logs, application logs, social media feeds
- Reliable delivery: guarantees at-least-once delivery using transactions between Source, Channel and Sink
- Flexible and extensible: plug in custom sources and sinks for any data format or destination

Stream Ingestion — Apache Kafka

Kafka is a distributed event streaming platform — a high-throughput, fault-tolerant, publish-subscribe messaging system built for real-time data pipelines.

Distributed

Runs as a cluster of brokers across multiple servers

Durable

Messages are persisted to disk and replicated across brokers

Scalable

Handles millions of messages per second through partitioning

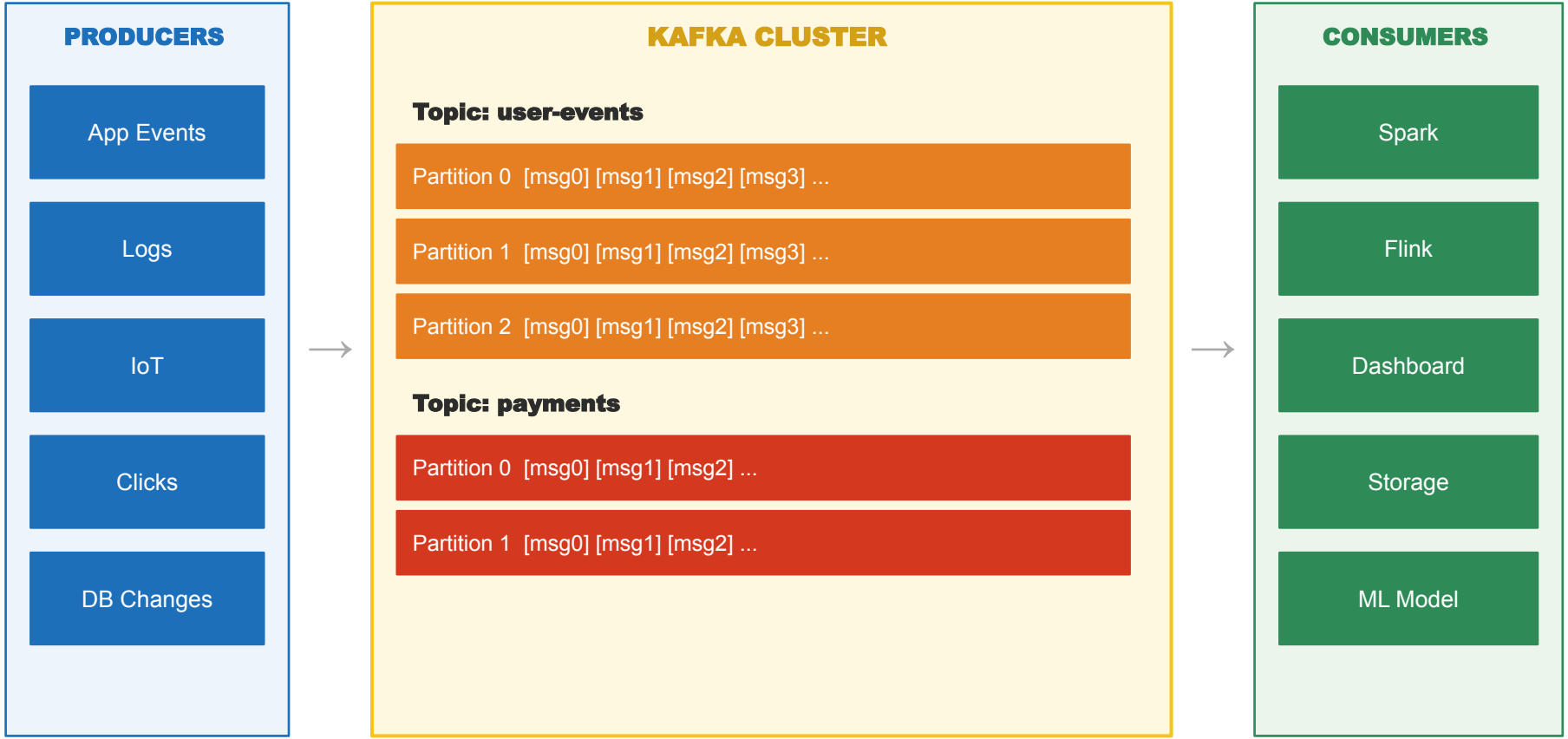
Replayable

Consumers can re-read past events — data is not lost on read

Core Concepts

- Topic: a named stream of records — like a category or feed. Producers write to topics, consumers read from them
- Partition: topics are split into partitions for parallelism. Each partition is an ordered, immutable sequence of records
- Offset: the position of a record within a partition. Consumers track their own offset — giving full control over replay
- Consumer Group: multiple consumers can share reading work across partitions for horizontal scalability

Kafka Architecture



Kafka + Spark Streaming — A Natural Pair

Kafka ingests the stream. Spark processes it. Together they power most real-time data pipelines in production.

Data Sources

Events · Logs
Sensors · Clicks

▶

Kafka

Buffer & distribute
the stream

▶

Spark Streaming

Process micro-batches
in near real-time

▶

Output

HDFS · DB
Dashboard · API

Why Kafka + Spark?

- Kafka acts as a buffer and decoupler — Spark reads at its own pace without overwhelming sources
- Fault tolerance — if Spark fails, it can resume from the last committed Kafka offset
- Kafka retains data — the same stream can be consumed by multiple Spark jobs independently
- Structured Streaming API in Spark makes Kafka integration straightforward with just a few lines of code

Apache NiFi — Data Flow Automation

NiFi is a visual, drag-and-drop data flow tool that automates the movement and transformation of data between systems — no coding required.

Key Characteristics

Visual Interface

Build data pipelines by dragging processors onto a canvas — no coding needed

Wide Connectivity

150+ built-in processors for HTTP, HDFS, Kafka, S3, databases, REST APIs and more

Data Provenance

Full lineage tracking — know exactly where every piece of data came from and where it went

Back Pressure

Automatically slows down sources if destinations can't keep up — preventing data loss

NiFi vs Kafka: NiFi excels at routing and transforming data between heterogeneous systems. Kafka excels at high-throughput event streaming. They are often used together.

Ingestion Layer — Summary

Choose your ingestion tool based on the nature of your data and your latency requirements

Tool	Type	Best For
Apache Sqoop	Batch	Historical loads, migrations
Apache Flume	Batch / Near-Real-Time	Log pipelines, web analytics
Apache Kafka	Streaming	Real-time pipelines, event-driven
Apache NiFi	Batch + Stream	System integration, IoT

Next: Storage Layer — where does all this ingested data go?

Section 3

Storage Layer

The Problem of Storing Data at Scale

🎭 From Distributed Storage to Data Swamps

It's 2003. Google publishes a paper on the Google File System — a way to store massive files across thousands of cheap commodity machines. A year later, Doug Cutting reads it and thinks: this is exactly what I need for my search engine project. He builds HDFS. He names the whole project Hadoop after his son's yellow toy elephant.

For years, HDFS is the gold standard. Companies dump everything into it — raw logs, JSON files, Avro, Parquet, images, you name it. The promise is simple: store everything, figure out structure later. They call it a Data Lake.

But years pass. The lake fills up. Nobody knows what's in it. Data arrives broken and nobody notices. A job fails silently and corrupt data flows downstream. The lake becomes a swamp.

Engineers at Databricks ask: what if we add a reliability layer on top? ACID transactions, schema enforcement, time travel. They build Delta Lake. Others follow with Apache Iceberg. The era of the Data Lakehouse begins.

The Storage Problem

Traditional storage systems were designed for structured data on a single machine. Big Data broke all those assumptions.

Volume

Single servers can no longer hold petabytes of data — we need to spread storage across many machines

Variety

Data is no longer just rows and columns — it's JSON, logs, images, videos, sensor readings and more

Reliability

If one machine fails, data should not be lost — replication and fault tolerance become critical

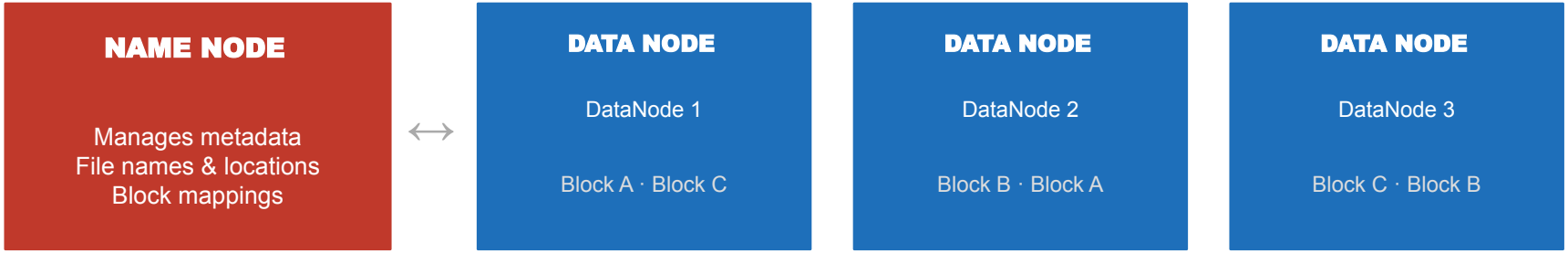
Cost

Enterprise storage is expensive — commodity hardware and cloud storage democratized the economics

HDFS — The Foundation of Distributed Storage

Hadoop Distributed File System — splits large files into blocks, distributes them across a cluster, and replicates each block for fault tolerance.

Architecture



Key Characteristics

- Default replication factor of 3 — each block is stored on 3 different DataNodes for fault tolerance
- Write once, read many — files are immutable after creation, optimized for large streaming reads
- Block size of 128MB by default — far larger than a typical filesystem, minimizing NameNode metadata overhead
- Rack awareness — Hadoop places replicas on different racks to survive full rack failures

NoSQL Databases — Beyond Relational Storage

When data is too big, too fast, or too varied for a relational database — NoSQL steps in.

Document

MongoDB
CouchDB

User profiles, catalogs, content management — semi-structured data with nested fields

Key-Value

Redis
DynamoDB

Caching, session management, leaderboards — ultra-fast lookup by a unique key

Wide-Column

HBase
Cassandra
Scylla

Time-series data, IoT, write-heavy workloads — billions of rows with flexible columns

Graph

Neo4j
JanusGraph

Social networks, recommendation engines, fraud detection — relationships are first-class

NoSQL ≠ No SQL. Many NoSQL databases offer SQL-like query interfaces. The name means 'Not Only SQL'.

Cloud Storage — Storage for Everyone

Cloud providers made distributed, scalable storage accessible to organizations of all sizes — no hardware needed.

Amazon AWS

Amazon S3

Simple Storage Service — object storage with 99.999999999% durability. Industry standard for data lakes.

Microsoft Azure

Azure Blob Storage

Massively scalable object storage. Azure Data Lake Storage Gen2 adds hierarchical namespace for big data analytics.

Google Cloud

Google Cloud Storage

Unified object storage with strong consistency. Integrates natively with BigQuery and Dataflow.

💡 Key insight: Cloud storage decouples compute from storage — you pay only for what you use, and you scale them independently.

Columnar Storage — Parquet & ORC

How you store data matters as much as where you store it. Columnar formats revolutionized analytics performance.

Row Storage (Traditional)

ID	Name	Age	City
1	Alice	28	Paris
2	Bob	34	Berlin
3	Carol	25	Tokyo

Reading 'Age' column requires scanning ALL rows

Columnar Storage (Parquet / ORC)

ID	Name	Age	City
1	Alice	28	Paris
2	Bob	34	Berlin
3	Carol	25	Tokyo

Reading 'Age' column reads ONLY that column's data

Benefits of Columnar Storage

- Faster analytics queries — reads only the columns needed, skipping irrelevant data entirely
- Superior compression — values in the same column are often similar, enabling much higher compression ratios
- Parquet is the de facto standard for big data — used by Spark, Hive, Presto, Athena and virtually every modern tool

Data Lake vs Data Warehouse

Two fundamentally different philosophies for storing and organizing data at scale.

Attribute	Data Lake 🏞️	Data Warehouse 🏛️
Data Structure	Raw, unstructured, any format	Structured, schema-on-write
Schema	Schema-on-read (defined at query time)	Schema-on-write (defined upfront)
Users	Data scientists, engineers	Business analysts, executives
Cost	Low (commodity / cloud storage)	High (specialized hardware / licensing)
Query Speed	Slower (raw files)	Fast (optimized indexes)
Examples	HDFS, Amazon S3, Azure Data Lake	Snowflake, BigQuery, Redshift

Delta Lake — Reliability on Top of the Lake

Delta Lake is an open-source storage layer built by Databricks that brings ACID transactions, schema enforcement, and time travel to data lakes stored on S3, ADLS, or HDFS.

Why Delta Lake was needed

⚡ ACID Transactions

Multiple writers can safely write to the same table simultaneously. Partial writes and corrupt data become a thing of the past.

Schema Enforcement

Reject writes that don't match the expected schema — no more silently ingested corrupt data flowing through your pipeline.

⏰ Time Travel

Query any previous version of your data. Made a mistake? Roll back to any point in history. Every write creates a new version.

Upserts & Deletes

Supports MERGE, UPDATE and DELETE operations on immutable file storage — critical for GDPR compliance and CDC patterns.

Delta Lake is the foundation of the Databricks Lakehouse Platform — and one of the most widely adopted open-source storage formats today.

Apache Iceberg — The Open Table Format

Apache Iceberg is an open table format for huge analytic datasets — designed to fix the problems of Hive-style tables and work at the scale of Netflix, Apple and LinkedIn.

What makes Iceberg different?

- Hidden partitioning — Iceberg handles partitioning automatically so users never need to write WHERE dt=... clauses
- Snapshot isolation — readers always see a consistent snapshot even while writers are committing new data
- Schema evolution without rewriting — add, rename or drop columns without rewriting the entire table
- Engine agnostic — works with Spark, Flink, Trino, Hive, Dremio and virtually every modern query engine

Delta Lake vs Apache Iceberg

Feature	Delta Lake	Apache Iceberg
Open Source	✅ Yes (Linux Foundation)	✅ Yes (Apache Foundation)
Primary Engine	Spark / Databricks	Engine agnostic (Spark, Flink, Trino...)
Best Known For	Streaming + batch unification	Hidden partitioning + large scale

Data Lakehouse — Best of Both Worlds

What if you could have the flexibility of a data lake and the reliability of a data warehouse in one architecture?

Data Lake 🏞️

- ✅ Low cost storage
- ✅ Any data format
- ✅ Scalable

- ❌ No reliability
- ❌ Slow queries
- ❌ No transactions

Data Lakehouse 🏠

- ✅ Low cost storage
- ✅ ACID transactions
- ✅ Fast analytics
- ✅ Schema enforcement
- ✅ BI + ML in one

Data Warehouse 🏛️

- ✅ Fast queries
- ✅ ACID transactions
- ✅ Strong governance

- ❌ Expensive
- ❌ Structured only
- ❌ Rigid schema

Implemented with: Delta Lake · Apache Iceberg · Apache Hudi

Storage Layer — Summary

The storage landscape has evolved from single servers → distributed files → smart tables with reliability guarantees

2003–2010

HDFS

Distributed
file storage

▶

2010–2015

NoSQL DBs

Document, KV
Wide-column, Graph

▶

2015–2018

Data Lake

S3 / ADLS
Raw everything

▶

2018–Now

Lakehouse

Delta / Iceberg
ACID + Scale

Key Takeaways

- Use Parquet or ORC for all analytical workloads — the performance gains are massive
- Delta Lake or Iceberg gives you data lake economics with data warehouse reliability
- Cloud storage (S3, ADLS, GCS) is the new default for scalable, cost-effective data storage

Next: Processing Layer — how do we make sense of all this stored data?

Section 4

Processing Layer

The Problem of Making Data Useful

🎭 The Story of Spark — Why Move Data to RAM?

MapReduce works. But it's painfully slow. Every time you process data, you read from disk, write intermediate results back to disk, read again, write again. For iterative algorithms like machine learning, you repeat this cycle hundreds of times.

It's 2009. A graduate student at UC Berkeley's AMPLab named Matei Zaharia is staring at this problem. He asks a simple question: why are we moving the computation to the data sitting on disk? What if instead, we move the data into RAM and keep it there across multiple operations?

He starts writing code. The insight is elegant — keep data in memory as a distributed dataset, apply transformations lazily, and only compute when you need results. He calls it Spark.

The results are staggering. Spark runs up to 100x faster than MapReduce in memory. It is open-sourced in 2010, donated to Apache in 2013, and by 2014 it becomes the most active open-source big data project in the world. Meanwhile in Germany, another team builds Apache Flink for true real-time stream processing.

The Processing Problem

Once data is stored, making sense of it at scale is the hardest engineering challenge of all.

Two Processing Paradigms

BATCH PROCESSING

- Process large chunks of historical data at scheduled intervals
- High throughput, lower latency requirements
- Tools: MapReduce, Spark batch, Hive
- Use cases: daily reports, ETL pipelines, model training
- Example: calculate all user engagement metrics for yesterday at 2am

VS

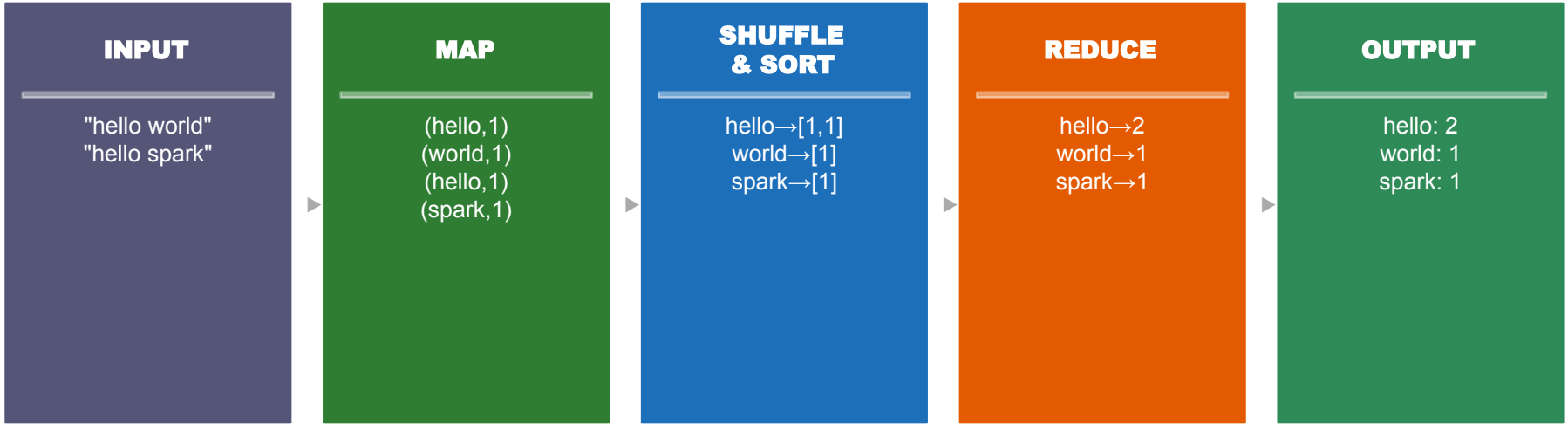
STREAM PROCESSING

- Process individual events or micro-batches as they arrive
- Low latency, results in milliseconds to seconds
- Tools: Spark Streaming, Apache Flink, Kafka Streams
- Use cases: fraud detection, live dashboards, real-time recommendations
- Example: flag a fraudulent transaction within 50ms of it happening

MapReduce — Where It All Started

MapReduce is a programming model for processing massive datasets in parallel across a cluster. Two phases: Map (transform) and Reduce (aggregate).

Classic Example: Word Count



Why MapReduce became a bottleneck

- Every Map and Reduce step reads from and writes back to HDFS — massive disk I/O overhead for iterative jobs
- No support for streaming, interactive queries, or machine learning workflows out of the box

Apache Spark — The Unified Processing Engine

Spark is a unified, in-memory distributed computing engine. One engine to rule batch, streaming, SQL, machine learning and graph processing.

Spark SQL

Structured data & SQL queries

Spark Streaming

Real-time stream processing

MLlib

Machine learning algorithms

GraphX

Graph computation & analysis

Apache Spark Core — In-Memory Distributed Computation Engine

Key Advantages over MapReduce

- Up to 100x faster in memory, 10x faster on disk — by eliminating the repeated read/write cycles to HDFS
- Lazy evaluation — transformations are not executed until an action is triggered, enabling smart optimization
- APIs in Python, Scala, Java and R — making it accessible to data scientists and engineers alike
- Unified engine — one platform for all workloads instead of separate tools for each use case

Spark Deep Dive — RDDs, DataFrames & Datasets

Spark has evolved its core data abstraction three times — each generation more powerful and easier to use.

RDD

Spark 1.x (2010)

Resilient Distributed Dataset

The original Spark abstraction. A fault-tolerant, immutable distributed collection of objects. Gives full control but requires manual optimization.

✅ Full flexibility, fine-grained control

⚠️ Verbose API, no built-in optimization, harder to use

DataFrame

Spark 1.3 (2015)

Distributed Table

A distributed table with named columns — like a SQL table or pandas DataFrame. Spark's Catalyst optimizer automatically optimizes queries.

✅ SQL-like API, automatic optimization, fast

⚠️ No compile-time type safety (Python/R)

Dataset

Spark 1.6 (2016)

Typed DataFrame

Combines the type safety of RDDs with the optimization of DataFrames. Best of both worlds — but only available in Scala and Java.

✅ Type-safe, optimized, object-oriented API

⚠️ Scala/Java only — Python uses DataFrames

💡 *In practice: use DataFrames in Python/R, use Datasets in Scala/Java, avoid RDDs unless you need low-level control.*

Spark SQL — Query Everything with SQL

Spark SQL lets you query structured data using standard SQL — whether it lives in Parquet files, Hive tables, JSON, databases or Delta Lake.

Catalyst Optimizer

Automatically rewrites and optimizes SQL queries — chooses the best join strategy, pushes filters down, prunes columns.

Tungsten Engine

Generates optimized bytecode at runtime, manages memory off-heap, and uses SIMD CPU instructions for blazing performance.

Unified Data Access

Query Parquet, JSON, ORC, Avro, JDBC, Delta Lake and Hive tables with the same SQL — Spark handles the differences.

Hive Compatibility

Spark SQL supports HiveQL syntax and can read Hive metastore tables — making migration from Hive to Spark seamless.

```
# Register DataFrame as a SQL temp view and query it
df.createOrReplaceTempView("sales")
result = spark.sql("SELECT region, SUM(revenue) FROM sales WHERE year=2024 GROUP BY region")
```

Spark Streaming & Structured Streaming

Spark extends its batch engine to handle real-time data streams — treating a stream as an unbounded table that keeps growing.

Two Generations of Spark Streaming

DStream API (Legacy)

- Discretized Streams — divides stream into micro-batches of RDDs
- Works but limited: no event-time support, hard to handle late data
- Legacy approach — still works but not recommended for new projects

Structured Streaming (Current)

- Treats the stream as an unbounded DataFrame — use the same SQL/DataFrame API
- Full support for event-time windows, watermarks, and late arriving data
- Exactly-once guarantees and native Kafka/Delta Lake integration

Key Streaming Concepts

- Trigger interval: how often Spark processes a micro-batch — from 0ms (continuous) to minutes
- Watermark: tells Spark how late data can arrive before it is dropped — balances correctness vs latency
- Output modes: Append (new rows only), Update (changed rows), Complete (full result table)

Spark MLlib — Machine Learning at Scale

MLlib brings scalable machine learning directly into the Spark ecosystem — train models on terabytes of data across a cluster.

Classification

Logistic Regression
Decision Tree
Random Forest
Gradient Boosted Trees
Linear SVM
Naïve Bayes

Regression

Linear Regression
Generalized Linear
Decision Tree
Random Forest
Gradient Boosted

Clustering

K-Means
Gaussian Mixture
LDA (Topic Modeling)
Bisecting K-Means

Recommendation

ALS (Collaborative Filtering)
Matrix Factorization

ML Pipelines API

MLlib's Pipeline API chains Transformers and Estimators — mimicking scikit-learn's pipeline concept for reproducible, deployable ML workflows.

Spark Optimization & Tuning

Spark is fast by default — but knowing these concepts separates good from great Spark engineers.

Partitioning

Data is split into partitions processed in parallel. Too few = underutilized cluster. Too many = scheduling overhead. Rule of thumb: 2–4 partitions per CPU core.

Caching & Persistence

Use .cache() or .persist() to store intermediate DataFrames in memory when you need to reuse them across multiple actions. Avoid recomputing the same data.

Avoiding Shuffles

Shuffle = data moving across the network — the most expensive operation in Spark. Use broadcast joins for small tables, partition wisely, and avoid groupBy on high-cardinality columns.

Broadcast Variables

Small datasets (lookup tables, configs) can be broadcast to all executors once rather than shipped with every task. Use spark.broadcast() to eliminate repeated data transfer.

Predicate Pushdown

Spark pushes WHERE filters down to the data source (Parquet, Delta) so only relevant data is read from disk. Use columnar formats to maximize this benefit.

Catalyst & Tungsten

Spark's query optimizer (Catalyst) rewrites your query plan automatically. Tungsten generates optimized JVM bytecode. Trust the optimizer — use DataFrames, not RDDs.

Spark GraphX — Graph Processing at Scale

GraphX extends Spark with an API for graphs and graph-parallel computation — treating vertices and edges as distributed collections.

What is a Graph in this context?

- A graph consists of vertices (nodes) and edges (relationships between nodes)
- Vertex example: a user profile, a product, a web page, a financial account
- Edge example: a friendship, a purchase, a link, a transaction between accounts

Real-World Use Cases

Social Networks

Friend recommendations, community detection, influence scoring — who knows whom and how strongly

Fraud Detection

Detect rings of fraudulent accounts connected by shared devices, addresses or transaction patterns

PageRank

Google's original algorithm — rank web pages by importance based on incoming links. Also used for entity importance scoring

Supply Chain

Model suppliers, warehouses, routes and dependencies as a graph to find bottlenecks and optimize logistics

Spark Architecture — How It All Works Together

Understanding Spark's architecture helps you write better code and diagnose performance issues.

DRIVER PROGRAM

SparkContext / SparkSession

- Creates execution plan
- Manages job scheduling
- Communicates with Cluster Manager
- Collects results

↔

CLUSTER MANAGER

YARN · Mesos
Kubernetes · Standalone

↓

EXECUTORS

Worker processes on each node
- Run tasks assigned by Driver
- Store data in memory/disk
- Multiple tasks in parallel

Execution Model

- 1. Your code creates a DAG (Directed Acyclic Graph) of transformations
- 2. DAG is optimized by Catalyst into a physical plan
- 3. Plan is split into Stages separated by shuffles
- 4. Each Stage becomes Tasks distributed to Executors
- 5. Results flow back to the Driver

Apache Flink — True Stream Processing

Apache Flink is a distributed stream processing framework that treats streaming as the primary paradigm — batch is just a special case of a bounded stream.

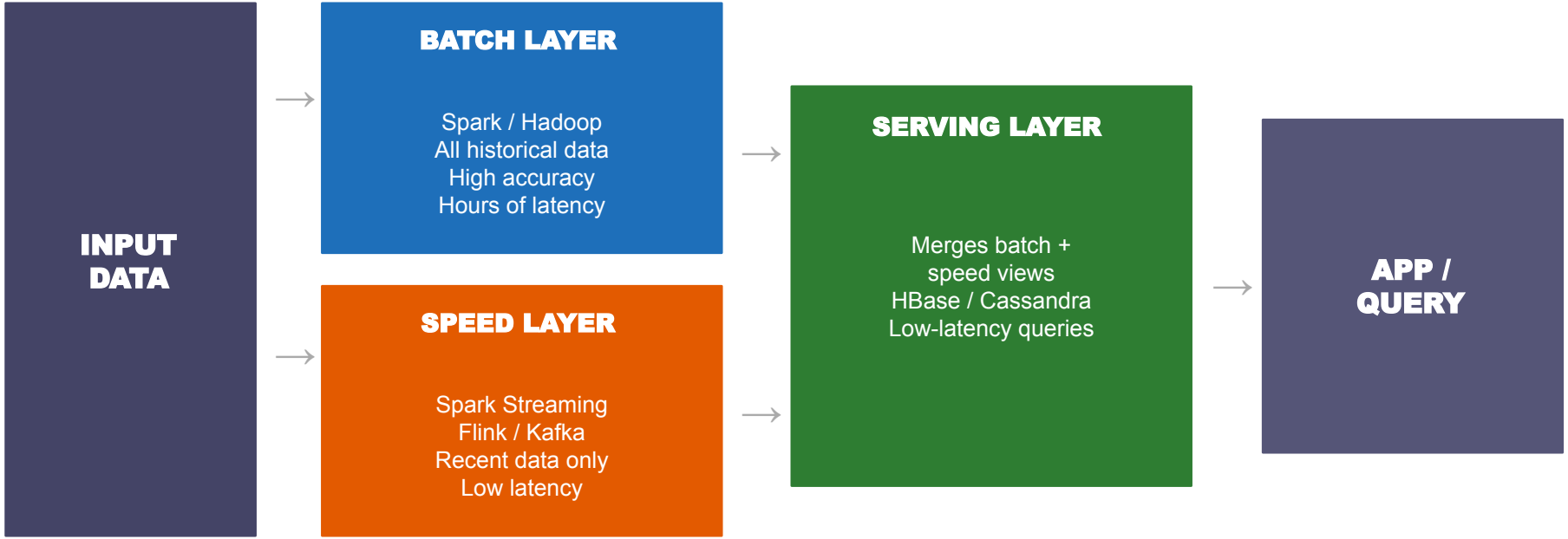
Spark Streaming vs Apache Flink

Attribute	⚡ Spark Streaming	🌊 Apache Flink
Processing Model	Micro-batch (near real-time)	True event-by-event streaming
Latency	Milliseconds to seconds	Sub-millisecond possible
State Management	Good with checkpointing	Excellent — native stateful operators
Event Time	Supported (Structured Streaming)	First-class citizen from day one
Batch Support	Primary strength	Supported (bounded streams)
Best For	Unified batch + streaming, ML	Ultra-low latency, complex event processing

💡 Choose Flink when you need true real-time with sub-second latency. Choose Spark when you need a unified platform for batch + streaming + ML.

Lambda Architecture — Batch + Speed + Serving

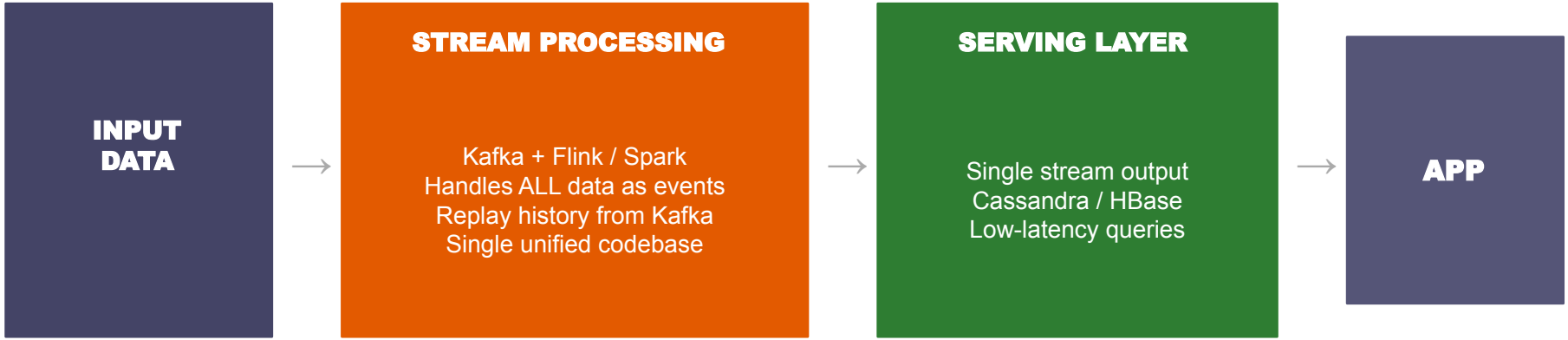
Lambda architecture solves the problem of combining accurate historical results with real-time data — using two parallel processing paths.



✅ Accurate historical results + real-time freshness ⚠️ Two codebases to maintain — complexity and operational overhead

Kappa Architecture — Everything is a Stream

Kappa simplifies Lambda by eliminating the batch layer entirely — treating all data as a stream, including historical reprocessing.



How Kappa handles historical data — the key insight

- Kafka retains all events for a configurable retention period — days, weeks, or forever
- To reprocess history: spin up a new version of the stream job, replay from the beginning of Kafka topic
- Once the new job catches up to present, swap the old serving layer — zero downtime reprocessing

✅ Single codebase, simpler operations ⚠️ Requires long Kafka retention and sufficient compute for replay

Lambda vs Kappa — Choosing the Right Architecture

Both architectures solve the same problem — the right choice depends on your team, data, and latency requirements.

Factor	Λ Lambda	κ Kappa
Complexity	High — two systems to build, test, deploy	Lower — one unified streaming system
Latency	Near real-time + accurate batch results	Real-time with eventual historical accuracy
Reprocessing	Rerun batch layer on historical data	Replay Kafka from offset — elegant
Cost	Higher — running two processing paths	Lower — one path, but needs Kafka storage
Team Skill	Needs batch + streaming expertise	Streaming expertise only
Best For	Systems where batch accuracy is critical	Systems with high event volumes + replay needs

💡 Industry trend: Kappa is gaining adoption as Kafka retention costs drop and streaming tooling matures. Many teams that started with Lambda are migrating to Kappa.

Processing Layer — Summary

Spark is the Swiss Army knife of big data processing — but knowing when to use Flink and which architecture pattern matters just as much.

MapReduce

Foundation

The original paradigm — understand it conceptually but use Spark in practice

Apache Spark

Unified Engine

Batch, SQL, streaming, ML, graphs — one engine for all workloads

Apache Flink

True Streaming

Sub-millisecond latency, stateful stream processing, CEP

Lambda

Architecture Pattern

Batch accuracy + real-time speed — two layers merged at serving

Kappa

Architecture Pattern

Stream-first simplicity — everything replayed from Kafka

Next: Modern Data Engineering — dbt, Airflow, and the Modern Data Stack

Section 5

Modern Data Engineering

dbt · Apache Airflow · The Modern Data Stack

🎭 The Story of dbt & Airflow — Engineering Discipline Meets Data

It's 2016. A small analytics team at a startup called Fishtown Analytics is buried in SQL scripts. Hundreds of them. Nobody knows which one runs first. Nobody can test them. Nobody can version them. A failed transformation silently corrupts a dashboard and the CEO makes a decision based on wrong numbers.

One engineer asks: why can't we treat SQL transformations like software? With version control, tests, documentation, and dependency management? He builds dbt — data build tool. It turns SQL SELECT statements into a full software engineering workflow.

Meanwhile at Airbnb in 2014, data engineers are manually triggering pipelines, chasing failures at 3am, and managing dependencies between jobs with spreadsheets. Maxime Beauchemin asks: why can't workflows be code? He builds Airflow — where pipelines are defined as Python DAGs, scheduled automatically, and monitored in a beautiful UI.

Together, dbt and Airflow form the backbone of what the industry now calls the Modern Data Stack — a new philosophy where data engineering borrows the discipline and tooling of software engineering.

ETL vs ELT — A Fundamental Shift

The rise of cloud data warehouses flipped the traditional data pipeline pattern — and opened the door for dbt.

Traditional ETL

- Transformation happens before loading — requires a separate ETL tool (Informatica, Talend, SSIS)
- Expensive and rigid — changing a transformation means rebuilding the pipeline

Modern ELT (with dbt)

- Raw data lands in warehouse first — transformation happens inside using SQL (dbt)
- Cloud warehouses (Snowflake, BigQuery, Redshift) are powerful enough to handle transformations at scale
- dbt brings version control, testing, and documentation to SQL transformations

💡 ELT is only possible because modern cloud warehouses are powerful enough to run transformations at massive scale cost-effectively.

dbt — Data Build Tool

dbt is a transformation tool that lets data analysts and engineers write modular SQL SELECT statements, then dbt handles materializing them as tables or views in the warehouse.

Core Philosophy: Treat Data Transformations Like Software

🔗 Modularity & Dependencies

Models reference each other using ref() — dbt automatically builds a DAG and executes in the right order. No more guessing which script runs first.

✅ Built-in Testing

Write tests in YAML: not_null, unique, accepted_values, relationships. Run dbt test to validate data quality on every deployment.

📚 Auto-Documentation

dbt generates a data catalog from your models automatically — every table, every column, every relationship documented and searchable.

🔄 Version Control Native

dbt projects live in Git. Every SQL change is reviewed, versioned, and reversible — the same workflow as application code.

🏭 Materializations

Choose how models are built: table (full rebuild), view (virtual), incremental (append/merge new rows), ephemeral (no storage — just a CTE).

☁️ Warehouse Agnostic

Works with Snowflake, BigQuery, Redshift, Databricks, DuckDB, Postgres and more — write SQL once, dbt handles dialect differences.

dbt — Project Structure & Model Layers

A well-structured dbt project follows a layered architecture — each layer serves a specific purpose in the transformation pipeline.

SOURCES

Raw tables loaded by ingestion tools (Fivetran, Airbyte)

`{{ source('stripe', 'payments') }}`

STAGING

1:1 with source
Rename columns
Cast types
Light cleaning

`stg_stripe__payments.sql`

INTERMEDIATE

Business logic
Joins & aggregates
Reusable building blocks

`int_orders_enriched.sql`

MARTS

Business-facing fact & dim tables
Ready for BI tools and analytics

`fct_orders.sql`

A dbt model is just a SQL SELECT statement:

```
-- models/marts/fct_orders.sql
SELECT o.order_id, o.created_at, c.customer_name, SUM(i.amount) AS total
FROM {{ ref('stg_orders') }} o JOIN {{ ref('int_customers') }} c USING (customer_id)
```

dbt — Testing & Documentation

dbt brings software engineering best practices — testing and documentation — directly into the data transformation workflow.

Data Testing

```yaml
# schema.yml
models:
  - name: fct_orders
    columns:
      - name: order_id
        tests:
          - not_null
          - unique
      - name: status
        tests:
          - accepted_values:
              values: ['placed','shipped']
```

- Generic tests: not_null, unique, accepted_values, relationships — declare in YAML
- Custom tests: write any SQL — if it returns rows, the test fails
- Run dbt test after every deployment — catches data quality issues early

Auto-Documentation

📊 Lineage Graph

dbt generates a visual DAG showing how every model depends on every other model — full data lineage at a glance

📝 Column Descriptions

Add descriptions to models and columns in YAML — dbt generates a searchable HTML catalog automatically

🔍 dbt docs serve

One command spins up a web server with your full data dictionary — shareable across the entire data team

Apache Airflow — Workflows as Code

Airflow is a workflow orchestration platform that lets you define, schedule, and monitor data pipelines as Python code — using Directed Acyclic Graphs (DAGs).

Core Concepts

DAG

Directed Acyclic Graph

A collection of tasks with defined dependencies — the graph is acyclic so there are no circular dependencies or infinite loops

Task

Unit of Work

A single step in a pipeline — could be a Python function, a SQL query, a Spark job, an API call, or an email notification

Operator

Task Template

Pre-built task types: PythonOperator, BashOperator, SparkSubmitOperator, BigQueryOperator, HttpOperator and hundreds more

Scheduler

Pipeline Trigger

Parses DAG files, determines which tasks are ready to run, and places them in the task queue based on schedule and dependencies

Airflow was built at Airbnb (2014), open-sourced in 2015, donated to Apache in 2016 — now the de facto standard for data pipeline orchestration.

Airflow Architecture

Airflow is a distributed system with several components working together to schedule and execute workflows reliably.

Web Server

Provides the Airflow UI — monitor DAG runs, view logs, manually trigger or pause DAGs, and inspect task states

Scheduler

Continuously monitors DAGs, determines task readiness, and sends executable tasks to the Queue

Executor + Queue

Receives tasks from Scheduler and distributes them to Workers. Types: Local, Celery, Kubernetes

Metadata Database

PostgreSQL or MySQL — stores DAG definitions, task states, run history, variables, and connections. The single source of truth for Airflow.

Workers

Execute individual tasks. Can be scaled horizontally. Each worker picks tasks from the queue and reports results back to the metadata DB.

DAG files are typically stored in Git and synced to the Scheduler — GitSync is the recommended pattern for production.

Airflow — Writing a DAG

A DAG is a Python file. Tasks are defined as operators and wired together with >> (downstream) dependencies.

```python
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.apache.spark.operators.spark_submit \
    import SparkSubmitOperator
from datetime import datetime

with DAG(
    dag_id="daily_sales_pipeline",
    schedule_interval="0 6 * * *",  # Every day at 6am
    start_date=datetime(2024, 1, 1),
    catchup=False
) as dag:

    extract = PythonOperator(
        task_id="extract_from_api",
        python_callable=extract_sales_data
    )

    transform = SparkSubmitOperator(
        task_id="transform_with_spark",
        application="jobs/transform.py"
    )

    load = PythonOperator(
        task_id="load_to_warehouse",
        python_callable=load_to_bigquery
    )

    extract >> transform >> load
```

What this DAG does

① **Schedule**
Runs every day at 6am via cron expression

② **Extract Task**
Calls a Python function to pull sales data from an API

③ **Transform Task**
Submits a Spark job to process and clean the data

④ **Load Task**
Loads the cleaned data into BigQuery warehouse

>> **Dependencies**
Tasks run in order: extract must succeed before transform, transform before load

>> is Airflow's syntax for 'this task must run before that task'

The Modern Data Stack

The Modern Data Stack (MDS) is a collection of cloud-native, best-of-breed tools that work together to form a complete data platform.

Data Sources	Databases · APIs · SaaS Apps · Events · Files
Ingestion	Fivetran · Airbyte · Stitch · Kafka · custom scripts
Storage	Snowflake · BigQuery · Redshift · Databricks · DuckDB
Transformation	dbt · Spark · Dataform
Orchestration	Apache Airflow · Prefect · Dagster

Consumption & Visualization

BI & Dashboards

Tableau · Power BI · Looker · Metabase · Superset

Data Science / ML

Jupyter · MLflow · SageMaker · Vertex AI

Reverse ETL

Census · Hightouch — push data back to CRMs & tools

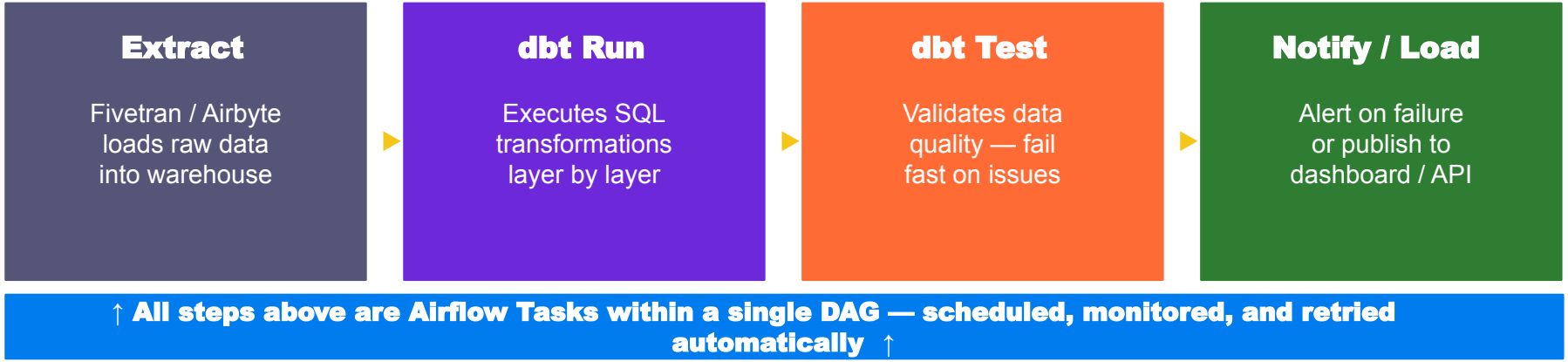
APIs

REST / GraphQL endpoints served from the warehouse

Data Governance & Observability — Monte Carlo · Great Expectations · Collibra · dbt tests

dbt + Airflow — The Power Couple

dbt handles the what (SQL transformations). Airflow handles the when and how (orchestration, scheduling, dependencies).



Why they work so well together

- dbt knows about model dependencies within transformations — Airflow knows about pipeline dependencies across systems
- Airflow's DbtRunOperator and DbtTestOperator make running dbt commands inside Airflow DAGs seamless
- dbt Cloud offers a native Airflow integration — trigger dbt jobs via API and monitor results in Airflow UI
- Together they give you end-to-end pipeline observability — from raw source data all the way to tested, documented marts

Modern Data Engineering — Summary

The shift is cultural as much as technical — data teams are adopting software engineering discipline, and the tooling reflects that.

ETL → ELT

Paradigm Shift

Load raw data first, transform inside the warehouse using SQL — enabled by cheap cloud compute

dbt

Transformation Layer

SQL-first transformations with version control, testing, documentation and modular DAG-based dependencies

Apache Airflow

Orchestration Layer

Python-defined DAGs that schedule, monitor, and retry complex multi-system data pipelines automatically

Modern Data Stack

Philosophy

Cloud-native, best-of-breed tools: ingest → store → transform → orchestrate → visualize — each layer replaceable

Next: Visualization Layer — how do we make data accessible to humans?

Section 6

Visualization Layer

The Problem of Making Data Accessible to Humans

🎭 The Story of ELK — Never Designed as a Stack

It's 2010. A developer named Shay Banon is building a recipe search engine for his wife. She is studying to be a chef and needs a way to search her growing cookbook collection. He wraps Apache Lucene in a simple REST API, makes it distributed, and releases it as Elasticsearch — just a side project.

Engineers at companies discover it and start using it for something totally different — storing and searching logs. It is perfect: schema-flexible, fast, and distributed. But reading raw JSON from Elasticsearch is painful. They need a way to visualize it.

Kibana appears — a web interface that turns Elasticsearch data into charts, dashboards, and maps. Then the question becomes: how do we reliably ship logs into Elasticsearch? Logstash emerges as the data pipeline for collecting, transforming, and sending logs.

Nobody designed the ELK Stack. It assembled itself, tool by tool, around a shared problem. Today it is one of the most widely deployed observability stacks in the world — and the company behind it, Elastic, went public at a $2.5B valuation.

The Visualization Problem

Processed data sitting in a warehouse or Elasticsearch index has no value until a human can understand it and act on it.

Who consumes data — and how?

📊 Business Users

Need:

Interactive dashboards, self-service reports, no code

Tools:

Power BI · Tableau · Looker · Metabase · Superset

Need to explore data and build reports without writing SQL. Drag-and-drop interfaces, pre-built dashboards, and alerting on KPIs.

🔬 Data Scientists

Need:

Exploratory analysis, model evaluation, custom visualizations

Tools:

Jupyter · Matplotlib · Plotly · Seaborn · Observable

Need programmatic access to data and flexible plotting libraries. Python notebooks are the primary interface for exploration and experimentation.

🛠️ Engineers & Ops

Need:

Real-time monitoring, log search, alerting, anomaly detection

Tools:

Kibana · Grafana · Datadog · New Relic

Need live dashboards showing system health, log search for debugging, and automatic alerts when metrics cross thresholds.

The ELK Stack — Elasticsearch, Logstash & Kibana

ELK is the go-to stack for log analytics, full-text search, and real-time operational intelligence — now officially called the Elastic Stack.

DATA SOURCES

App logs
Server metrics
APM traces
Security events
Business data

BEATS

Lightweight data shippers:
Filebeat · Metricbeat
Packetbeat

LOGSTASH

Collect · Parse
Enrich · Filter
Transform · Route
to destinations

ELASTICSEARCH

Index & Store
Full-text search
Aggregations
Distributed
REST API

KIBANA

Dashboards
Discover (search)
Maps · Canvas
Alerts · ML

Key Facts

- Elasticsearch uses an inverted index for near-instant full-text search across billions of documents
- Beats are lightweight agents (no JVM) — ideal for shipping data from thousands of servers with minimal overhead
- Kibana's Discover interface lets you search logs using KQL (Kibana Query Language) — SQL-like but for documents

Elasticsearch — Search and Analytics Engine

Elasticsearch is a distributed, RESTful search and analytics engine built on Apache Lucene. It stores data as JSON documents and makes them searchable in near real-time.

Core Concepts

Index
A collection of documents — equivalent to a database table. e.g., logs-2024-01, products, users

Document
A single JSON record stored in an index. Every document has a unique _id and typed fields

Shard
An index is split into shards — each a self-contained Lucene index distributed across nodes for parallelism

Replica
A copy of a shard on a different node — provides fault tolerance and increases read throughput

Query Capabilities

Full-Text Search
`match, multi_match, query_string`

Term-Level Queries
`term, range, exists, wildcard`

Aggregations
`avg, sum, histogram, date_histogram`

Geo Queries
`geo_distance, geo_bounding_box`

```
GET /logs-*/_search  { "query": { "match": { "message": "error" } }, "aggs": { "by_host": { "terms": { "field": "host.keyword" } } } }
```

Kibana — Visualize and Explore Your Data

Kibana is the visualization and exploration interface for Elasticsearch — turning raw JSON documents into charts, maps, dashboards and alerts.

🔍 Discover

Search and filter raw documents in real time using KQL. Essential for log debugging and ad-hoc exploration of indexed data.

📊 Visualize & Dashboard

Build bar charts, line graphs, pie charts, data tables, heatmaps and metrics. Combine into interactive dashboards shared with the team.

🗺️ Maps

Geo-spatial visualization — plot events on a world map, show density heatmaps, and layer multiple data sources for location intelligence.

🔔 Alerting & Actions

Set threshold-based or anomaly-based alerts. Trigger actions — Slack notifications, email, PagerDuty, webhooks — when conditions are met.

🤖 ML & Anomaly Detection

Built-in machine learning for unsupervised anomaly detection — automatically learn normal patterns and flag unusual spikes or drops.

🛡️ Security & SIEM

Kibana powers Elastic SIEM — correlate security events, detect threats, visualize attack timelines, and investigate incidents.

Business Intelligence Tools

BI tools connect directly to data warehouses and let business users build dashboards and reports without writing SQL.

Tableau

Enterprise Standard

The original drag-and-drop BI leader. Exceptional visualization capabilities, deep analytics, strong community. On-premise and cloud.

✦ **Most powerful visualizations, trusted by enterprises**

Power BI

Microsoft Ecosystem

Microsoft's BI platform — tightly integrated with Azure, Excel, and Teams. Excellent value and ubiquitous in enterprise environments.

✦ **Best Microsoft integration, cost-effective, widely adopted**

Looker

Semantic Layer (Google)

Unique model-first approach using LookML — define business logic once, explore it everywhere. Acquired by Google in 2019.

✦ **Best semantic layer, consistent metrics across the organization**

Apache Superset

Open Source

Free, open-source BI platform originally built at Airbnb. Connects to any SQL database, supports 40+ chart types, no licensing cost.

✦ **Free, flexible, great for data-savvy teams, cloud-native**

Choosing a BI tool is often a political decision as much as a technical one — go with what your business users will actually adopt and use.

APIs — Data as a Service

Not all data consumers are human — applications, mobile apps, and external partners consume data programmatically through APIs.

Two Primary API Patterns for Data

REST API

- Resource-based: each endpoint represents a resource (GET /orders, POST /users)
- Standard HTTP verbs: GET, POST, PUT, PATCH, DELETE
- Returns fixed response shapes — client gets everything, even unused fields
- Cacheable, stateless, widely understood by all developers
- Best for: public APIs, simple data access, broad compatibility

GraphQL

- Query language: clients specify exactly what fields they need — no over/under-fetching
- Single endpoint — all queries and mutations go through /graphql
- Strongly typed schema — self-documenting, enables powerful tooling and IDE support
- Excellent for complex, nested data and rapidly evolving frontends
- Best for: internal APIs, mobile apps, complex relationship queries

💡 Many modern data platforms expose query results as APIs — tools like Hasura and PostgREST auto-generate REST/GraphQL APIs directly from database tables.

Visualization Layer — Summary

Choose your visualization tools based on who the consumers are — business users, engineers, or applications.

ELK Stack

For: Engineers & Ops

Elasticsearch · Logstash · Kibana · Beats

BI Tools

For: Business Users

Tableau · Power BI · Looker · Superset

Notebooks

For: Data Scientists

Jupyter · Plotly · Matplotlib · Observable

APIs

For: Applications & Partners

REST · GraphQL · Hasura · PostgREST

Key Principle: Great visualization is not about pretty charts — it's about giving the right people the right data in the right format at the right time. A dashboard nobody uses is worse than no dashboard at all.

Next: Putting It All Together — the complete picture

Section 7

Putting It All Together

End-to-End Architecture · Tool Selection · The Big Data Landscape

Real-World Example — E-Commerce Data Platform

Let's build the complete architecture for a modern e-commerce platform that processes millions of events per day.

The Business Requirements

📊 Daily business reporting

Revenue, orders, returns, customer LTV — accurate batch reports ready by 7am every morning

🎯 Live product recommendations

Show personalized recommendations based on current session browsing behavior in real time

📱 Mobile app data API

Mobile app needs order status, product catalog, and user profile data via a low-latency REST API

👔 Executive dashboards

C-suite needs real-time KPI dashboards — GMV, conversion rate, cart abandonment, regional performance

How do we architect a system that satisfies ALL of these requirements simultaneously?

→ Let's build it layer by layer on the next slide

E-Commerce Platform — Complete Architecture

DATA SOURCES

Web App | Mobile | POS | ERP/CRM | Marketing

INGESTION

- Apache Kafka (Event Stream)
- Fivetran / Airbyte (Batch ETL)
- Flume / NiFi (Logs)

STORAGE

- Amazon S3 (Data Lake)
- Delta Lake / Iceberg (Lakehouse)
- Elasticsearch (Search)

PROCESSING

- Spark Streaming (Real-time)
- Spark Batch (Daily ETL)
- dbt (Transform)
- MLlib

ORCHESTRATION

- Apache Airflow — Orchestrates ALL pipelines above

VISUALIZATION

- Power BI (Business)
- Kibana (Ops/Logs)
- REST API (Mobile App)
- Jupyter (Data Science)

Kafka stream → Spark Streaming (fraud detection, recommendations) → Results served via API in <100ms

S3 + Delta Lake → Spark Batch → dbt → Power BI (daily reports ready at 7am)

App logs → Kafka → Elasticsearch → Kibana (real-time ops monitoring and customer support search)

Choosing the Right Tool for the Right Job

No tool is universally best — the right choice depends on your data, team, latency requirements, and budget.

❓ Transformation inside the warehouse?

✅ Yes → dbt (SQL-first, versioned, tested)

🔄 No → Custom Spark jobs (for complex logic)

❓ Processing structured data at scale?

✅ Yes → Spark SQL + Parquet + Delta Lake

🔄 No → Spark RDDs or raw file processing

❓ Business users need dashboards?

✅ Yes → Power BI / Tableau / Looker / Superset

🔄 No → Jupyter notebooks (for data scientists)

❓ Data storage on cloud?

✅ Yes → S3/ADLS/GCS + Delta Lake or Iceberg

🔄 No → HDFS (on-premise legacy environments)

The best engineers don't pick their favourite tool — they pick the right tool for the problem at hand.

The Big Data Landscape Today

Ingestion (Batch)	Apache Sqoop · Apache Flume · Fivetran · Airbyte · Stitch
Ingestion (Stream)	Apache Kafka · Apache NiFi · Amazon Kinesis · Azure Event Hubs
Storage (Files)	HDFS · Amazon S3 · Azure ADLS · Google Cloud Storage
Storage (Tables)	Delta Lake · Apache Iceberg · Apache Hudi
Storage (NoSQL)	HBase · Cassandra · MongoDB · Redis · DynamoDB
Processing (Batch)	Apache Spark · MapReduce · Apache Hive · Presto / Trino
Processing (Stream)	Spark Streaming · Apache Flink · Kafka Streams · Apache Storm
Transformation	dbt · Apache Spark SQL · Dataform · SQLMesh
Orchestration	Apache Airflow · Prefect · Dagster · Luigi · Argo Workflows
Visualization / BI	Kibana · Tableau · Power BI · Looker · Superset · Grafana

The Big Data Ecosystem — Complete Picture

S2

INGESTION

Kafka · Sqoop · Flume · NiFi

S3

STORAGE

HDFS · Delta Lake · Iceberg · S3 · NoSQL

S4

PROCESSING

Spark · Flink · MapReduce · Lambda · Kappa

S5

MODERN DE

dbt · Airflow · Modern Data Stack

S6

VISUALIZATION

ELK · BI Tools · APIs

The 5 Most Important Takeaways from This Course

- Big Data is not just about size — it is about velocity, variety, and the need for distributed architectures that scale horizontally
- Every layer of the ecosystem solves a specific problem — master the problem first, then the tool becomes obvious
- Spark is the Swiss Army knife — but knowing when to reach for Flink, dbt, Kafka, or Elasticsearch is what separates junior from senior engineers
- The Modern Data Stack is a philosophy shift — treating data engineering with the same discipline, testing, and version control as software engineering
- The landscape changes fast — the tools you use today may be different in 5 years, but the principles of distributed storage and processing are timeless

Thank You

You now know the complete Big Data Ecosystem — from raw data ingestion all the way to business insights.

What to Explore Next

Hands-On Labs

Spin up a local Spark cluster, write a Kafka producer, build a dbt project on DuckDB

Cloud Platforms

Explore AWS EMR, Databricks, Google Cloud Dataflow, Azure HDInsight — managed big data services

Certifications

Databricks Certified Associate, Google Professional Data Engineer, AWS Certified Data Analytics

Deep Dives

Designing Data-Intensive Applications (Kleppmann) · The Data Engineering Cookbook · dbt documentation